Algorithms. mid-term exam

Dr. Havana Rika

23.12.2024

Exam length: 100 minutes

- One reference page is allowed, in addition to the attached fact-sheet.
- You can use draft papers, but you need to answer only on the exam booklet, in the provided space and only on the front pages. Be aware that the back side is not scanned.
- You can cite and rely (without providing a proof) on theorems and claims that were proven in class or in the homework.
- In every question or section marked by ♣, you can write "I don't know" and get one quarter of the score (rounded down).

G O O D L U C K!

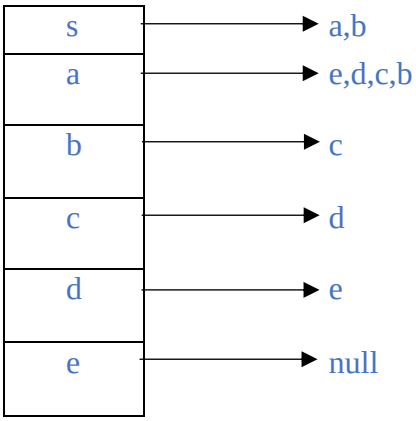
Question 1 (8*3 = 24 pts.)

For each of the following statements, determine if it is true or false. Justify by proving a true statement or providing a counter example for a false statement.

1. ♣ **1.1**. The height of every spanning tree resulting from DFS(s) on a directed unweighted simple graph G is equal to the length of a simple longest path from s to one of the other vertices in G.

   Reminder: the height of a tree is the length of a longest path from the root to one of its leaves.

   Answer: False. Consider the following graph



♣ **1.2**. Given a directed weighted graph G, we will define a new weight function w' by

$w'(e) = \alpha \cdot w(e)$ where α is some fixed positive constant. if P is a shortest path from u to v in G under the weight function w, then P is a shortest path from u to v in G under the weight function w'.

Answer: True.

We will use those definitions:

- w(P): The weight of path P under w, calculated as the sum of weights of edges in P.
- w′(P): The weight of path P under w', calculated as w′(P)=α·w(P).

If P is the shortest path from u to v under w, then: w(P)≤w(Q) for all paths Q from u to v.

Applying the transformation w′(e)=α·w(e), we get:

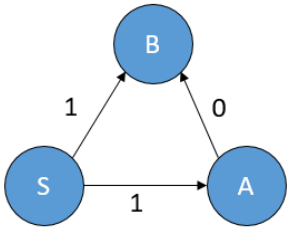
w′(P)=α·w(P), w′(Q)=α·w(Q).

Since α>0, the relationship w(P)≤w(Q) implies w′(P)≤w′(Q). Thus, P remains the shortest path under w′.

♣ 1.3 Given a weighted directed graph G with positive edge weights, i.e. w(e)>0 for every edge in G. Denote by $BF(v.d)$ and $BF(v.\pi)$ the values of $v.d$ and $v.\pi$ for every vertex v in G after an execution of Bellman-Ford on G. Similarly, denoted by $DIJ(v.d)$ and $DIJ(v.\pi)$ the values of $v.d$ and $v.\pi$ for every vertex v in G after an execution of Dijkstra algorithm on G. We argue that $BF(v.d) = DIJ(v.d)$ and $BF(v.\pi) = DIJ(v.\pi)$ must hold for every vertex v.

Answer: False. $BF(v.d) = DIJ(v.d)$ do holds, but it might be that $BF(v.\pi) \neq DIJ(v.\pi)$ if there is more than a single shortest path in G.

Consider the following graph:



Both paths S→B (directly) and S→A→B have the same shortest distance d(s,B)=1.

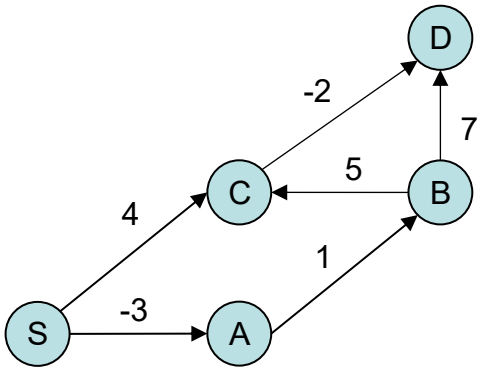
1. Dijkstra:
    a. Picks S→B as the shortest path (due to greedy selection).
    b. B.π=S
2. Bellman-Ford:
    a. Relaxes S→A first, then A→B.
    b. B.π=A

So:

- BF(B.d)=DIJ(B.d)=1 (shortest distance is correct).
- BF(B.π)≠DIJ(B.π).

Question 2 (26 pts.)

All the sections in this question refer to the weighted graph G=(V,E) in the figure. Each edge is labeled by its weight.



2.1 (10 pts.) Run Bellman-Ford Algorithm for finding shortest paths on the graph G with s as source.
The edges' order used in all sweeps is: SC, SA, AB, BD, CD, BC
Fill the table with the relax operations performed. Include only relax operations that cause updates of v.d values (the number of rows to be filled is the number of relax operation in which there is an update, it might be smaller than the number of rows in the empty table).

relax operation #	The edge on which relax is performed	The vertex v for which the value of v.d is decreased	The value of v.d after the relax	The relax is part of sweep number:
1	SC	C	4	1
2	SA	A	-3	1
3	AB	B	-2	1
4	BD	D	5	1
5	CD	D	2	1
6	BC	C	3	1
7	CD	D	1	2
8				
9				
10				

♣ **2.2 (8 pts.)** Suggest a different order of the edges, such that if this order is used, then all the shortest paths are updated correctly already in the first sweep. Shortly explain your answer.

To update all shortest paths in the first sweep, the new edge order should be:

SA, AB, BC, SC, CD, BD

This order ensures that all shortest paths are updated in the first sweep because the edges directly leading to shorter distances are relaxed first. This immediately propagates the shortest path information to all vertices

♣ **2.3 (8 pts.)** Suggest a different order of the edges, such that the maximum number of sweeps is needed in order to calculate correctly all the distances from s to the rest of the vertices. Shortly explain your answer.

To maximize the number of sweeps required to calculate the shortest paths, a possible order is:

CD, BC, AB, SC, BD, SA

This order maximizes sweeps. We saw in the lecture that if we relax edges in their reverse order along the shortest path, then the correct value will be updated after at least the number of edges on that path. Relaxing edges like BD or AB early delays updates to other vertices, as they don't directly affect the shortest path initially. This forces the algorithm to require more iterations (sweeps) to propagate the shortest path information throughout the graph.

♣ Question 3 (26 pts.)

Given is a directed graph $G = (V, E)$, and a source node $s \in V$. Each edge $e \in E$ is assigned a weight $w(e) \geq 0$ and a black or white color, i.e. $c(e) \in \{black, white\}$. A path P will be called "black-white path" if the **first** edge on this path is black and the **last** edge is white.
Suggest an efficient algorithm for computing for every node $v \in V$ the shortest "black-white path" from s to v. Describe the algorithm in words and formally prove its correctness (write the main claim and justify it in one paragraph), state and explain the time complexity.

Answer:

- **Construct three copies of the graph**:
  - $V_1 = \{v_1 \mid v \in V\}$, $V_2 = \{v_2 \mid v \in V\}$, $V_3 = \{v_3 \mid v \in V\}$.
  - $E_1 = \{(u_1, v_2) \mid (u, v) \in E \text{ and } c(u, v) = black\}$.
  - $E_2 = \{(u_2, v_2) \mid (u, v) \in E\}$.
  - $E_3 = \{(u_2, v_3) \mid (u, v) \in E \text{ and } c(u, v) = white\}$.
- **Create the weight function** w' on edges:
  - $w'(u_1, v_2) = w(u, v)$ for $e \in E_1$.
  - $w'(u_2, v_2) = w(u, v)$ for $e \in E_2$.
  - $w'(u_2, v_3) = w(u, v)$ for $e \in E_3$.
- **Run Dijkstra's algorithm** on the graph $G' = (V_1 \cup V_2 \cup V_3, E_1 \cup E_2 \cup E_3)$, starting from s_1. Retrieve the shortest path distance for each $v_3 \in V_3$ to compute the shortest black-white path for each $v \in V$.

Correctness:

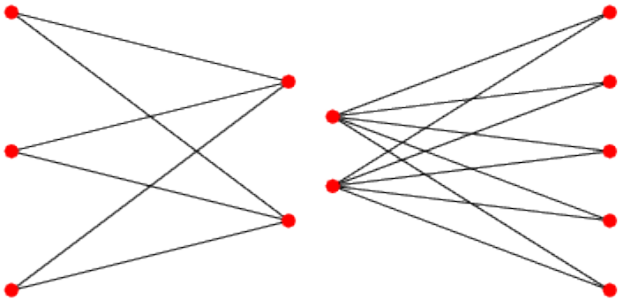
- **Main claim:** G' includes a path of length k from s_1 to v_3 if and only if G includes a black-white path of length k from s to v.
  In particular, the correctness of Dijkstra implies that $d(s_1, v_3)$ is the length of the shortest black-white path from s to v.
- **Justification**: The graph G' is constructed to enforce the structure of black-white paths. A path from s_1 to any node in V_3 must pass through V_2, as there are no edges directly connecting V_1 and V_3. All edges from V_1 to V_2 are black, and all edges from V_2 to V_3 are white, ensuring that any path includes a black edge first and a white edge last. (V_2, E_2) is a copy of , which guarantees the existence of any path in G.

Complexity:

- The number of nodes in G' is $3|V|$, and the number of edges is at most $3|E|$.
- Running Dijkstra's algorithm on G' has a time complexity of $\mathbf{O((3|E|) + (3|V| \log(3|V|))) = O(|E| + |V| \log |V|)}$.

Question 4 (8*3 = 24 pts.)

A undirected bipartite graph is an undirected graph whose vertices can be divided into two disjoint sets of vertices, V_0 and V_1, such that every edge connects a vertex in V_0 to one in V_1. Formally, we write $G = (V, E)$, where $V = V_0 \cup V_1$, $V_0 \cap V_1 = \emptyset$ and $E \subseteq V_0 \times V_1$.
A complete undirected bipartite graph, is one where every vertex in V_0 is connected to every vertex in V_1 (Formally: $E = V_0 \times V_1$).
For every $n_1, n_2 \geq 1$, K_{n_1,n_2} is the complete undirected bipartite graph with $|V_0| = n_1$ and $|V_1| = n_2$.
The following graphs are examples of $K_{3,2}$ (left) and $K_{2,5}$ (right):



Characterize the values of n_1 and n_2 for which K_{n_1,n_2} contains:

a. An Euler Cycle
b. An Euler Path
c. A Hamiltonian Cycle

Explain your answers.

Euler Cycle: A connected graph G has an Euler Cycle if and only if all of its vertices have even degree; it has an Euler path but no Euler Cycle if and only if it has exactly two vertices of odd degree. Each vertex in K_{n_1,n_2} has degree n_1 or n_2 (since every vertex in one side of the graph is connected to all vertices in the other side) so K_{n_1,n_2} has an Euler Cycle if and only if n_1 and n_2 are both even.

Euler Path:
K_{n_1,n_2} has an Euler path iff it has exactly two vertices of odd degree. This is true when:

- $n_1 = n_2 = 1$.
- n_1 is odd and $n_2 = 2$.
- n_2 is odd and $n_1 = 2$.

Hamilton Cycle: A path in K_{n_1,n_2} must alternate between vertices in V_0 and vertices in V_1. A cycle necessarily has $2k$ vertices for some positive integer k. k of these vertices are in V_0, and the other k are in V_1. Thus, if $n_1 \neq n_2$ it is impossible for a cycle in K_{n_1,n_2} to include every vertex, and therefore K_{n_1,n_2} can have a Hamilton circuit only if $n_1 = n_2$. It's easy to show by induction that $K_{n,n}$ has a Hamilton cycle for all $n \geq 2$.



Spare space – use if you need.